School ERP — Finance & Payroll Schema

Complete Table Report, Hierarchy & Entity-Relationship Diagram

---

A complete, final reference for the Payroll & Expenses schema: every table explained field by field with the reasoning behind each design decision, the three-tier structural hierarchy the schema follows, and a full entity-relationship diagram of all 13 tables.

Contents

Part 1 — Table-by-Table Report

Tables are listed in chronological build order — the order they must be created in, since later tables reference earlier ones.

Chronological build order:

- 1. employees
- 2. vendor_categories
- 3. vendors
- 4. directors
- 5. payroll_periods
- 6. payroll_entries
- 7. employee_advances
- 8. recurring_bills
- 9. vendor_transactions
- 10. director_draws
- 11. loans
- 12. users
- 13. audit_log

1. `employees`

```
CREATE TABLE employees (
    employee_id     SERIAL PRIMARY KEY,
    full_name       VARCHAR(150) NOT NULL,
    designation     VARCHAR(100),
    employment_type VARCHAR(30) DEFAULT 'permanent',
    bank_status     VARCHAR(20) DEFAULT 'online',
    bank_account_no VARCHAR(40),
    contact_no      VARCHAR(20),
    email           VARCHAR(150),
    is_active       BOOLEAN DEFAULT TRUE,
    created_at      TIMESTAMP DEFAULT now()
);
```

Purpose: master record of every staff member. Gives every employee a surrogate ID so nothing downstream ever has to store or match on a name.

Fields:

- employee_id — auto-incrementing surrogate key, referenced by payroll_entries, employee_advances, and (via party_id) loans.
- full_name — required.
- designation — job title (Teacher, Peon, Driver). Optional.
- employment_type — permanent/contract, defaults to 'permanent'.
- bank_status — online/cash/ob, defaults to 'online'.
- bank_account_no — optional, unvalidated format.
- contact_no, email — added mid-project for parity with vendors/directors.
- is_active — soft-delete flag; FALSE instead of a real delete, since payroll history references this row.
- created_at — auto-stamped via now().

Special concerns:

- No created_by/updated_by — that accountability layer only applies to transactional tables, not foundation/master tables.

2. `vendor_categories`

```
CREATE TABLE vendor_categories (
    category_id     SERIAL PRIMARY KEY,
    name            VARCHAR(60) UNIQUE NOT NULL,
    is_active       BOOLEAN DEFAULT TRUE
);
```

Purpose: a fixed, managed list of vendor categories (fuel, electrician, stationery, etc.), added specifically to stop free-text values like "Fuel"/"fuel "/"FUEL" from being treated as different categories.

Fields:

- category_id — surrogate key, referenced by vendors.category_id.
- name — UNIQUE (the first single-column uniqueness constraint in the schema) and required.
- is_active — soft-delete flag.

Special concerns:

- Deliberately has no created_at — closer to a fixed configuration list than a record worth tracing back to a creation date.

3. `vendors`

```
CREATE TABLE vendors (
    vendor_id       SERIAL PRIMARY KEY,
    name            VARCHAR(150) NOT NULL,
    category_id     INT REFERENCES vendor_categories(category_id),
    contact_no      VARCHAR(20),
    email           VARCHAR(150),
    bank_account_no VARCHAR(40),
    is_active       BOOLEAN DEFAULT TRUE,
    created_at      TIMESTAMP DEFAULT now()
);
```

Purpose: master record of every vendor (shop, contractor, supplier) the school pays. Same surrogate-key reasoning as employees.

Fields:

- vendor_id — surrogate key, referenced by vendor_transactions, recurring_bills, and (via party_id) loans.
- name — required.
- category_id — FK to vendor_categories; originally a free-text category column, normalized into its own table.
- contact_no, email — optional contact details.
- bank_account_no — added for parity with employees.
- is_active, created_at — standard patterns.

Special concerns:

- None outstanding — brought to full parity with employees and directors.

4. `directors`

```
CREATE TABLE directors (
    director_id     SERIAL PRIMARY KEY,
    full_name       VARCHAR(150) NOT NULL,
    contact_no      VARCHAR(20),
    email           VARCHAR(150),
    is_active       BOOLEAN DEFAULT TRUE,
    created_at      TIMESTAMP DEFAULT now()
);
```

Purpose: master record of the school's directors. Simplest of the three party tables.

Fields:

- director_id — surrogate key, referenced by director_draws and (via party_id) loans.
- full_name — required.
- contact_no, email — added mid-project; this table originally had only director_id, full_name, is_active.
- is_active, created_at — added for parity with employees and vendors.

Special concerns:

- None outstanding — brought to full parity with the other two party tables.

5. `payroll_periods`

```
CREATE TABLE payroll_periods (
    period_id       SERIAL PRIMARY KEY,
    period_month    SMALLINT NOT NULL CHECK (period_month BETWEEN 1 AND 12),
    period_year     SMALLINT NOT NULL,
    status          VARCHAR(20) DEFAULT 'open' CHECK (status IN ('open', 'closed')),
    UNIQUE (period_month, period_year)
);
```

Purpose: the first non-"party" table — represents a single calendar month of payroll. Everything about that month's payroll and director draws anchors to this row.

Fields:

- period_id — surrogate key, referenced by payroll_entries and director_draws.
- period_month — 1-12, enforced by CHECK. SMALLINT (smaller storage than INT, appropriate since the value never exceeds 12).
- period_year — no CHECK bound (no meaningful range to enforce).
- status — open/closed. CHECK added after discussing whether this should be a boolean instead; kept as text for consistency with employee_advances.status and room for a future third state.
- UNIQUE (period_month, period_year) — composite constraint; prevents duplicate months.

Special concerns:

- None outstanding — the missing CHECK on status was the one gap, now fixed.

6. `payroll_entries`

```
CREATE TABLE payroll_entries (
    entry_id            SERIAL PRIMARY KEY,
    employee_id         INT NOT NULL REFERENCES employees(employee_id),
    period_id           INT NOT NULL REFERENCES payroll_periods(period_id),
    days_worked         NUMERIC(4,1) NOT NULL DEFAULT 0,
    base_amount         NUMERIC(10,2) NOT NULL DEFAULT 0,
    security_deposit    NUMERIC(10,2) NOT NULL DEFAULT 0,
    security_deduction  NUMERIC(10,2) NOT NULL DEFAULT 0,
    advance_id          INT,
    advance_deducted    NUMERIC(10,2) NOT NULL DEFAULT 0,
    salary_paid         NUMERIC(10,2) NOT NULL DEFAULT 0,
    balance             NUMERIC(10,2) GENERATED ALWAYS AS (base_amount - salary_paid) STORED,
    notes               TEXT,
    UNIQUE (employee_id, period_id)
);
-- advance_id's FK is added via ALTER TABLE after employee_advances exists
```

Purpose: the first true transactional table — one employee's actual salary for one specific month. Where the "who" (employees) and "when" (payroll_periods) foundation tables actually meet and produce a real financial record.

Fields:

- entry_id — surrogate key.
- employee_id, period_id — required FKs.
- days_worked — up to 1 decimal place.
- base_amount — gross salary for the month.
- security_deposit, security_deduction — NOT NULL added mid-project for consistency, even though neither feeds the generated balance directly.
- advance_id — real FK to employee_advances, added via a separate ALTER TABLE (not inline) since payroll_entries is defined before employee_advances in the file.
- advance_deducted — the amount; now paired with advance_id so the deduction is traceable.
- salary_paid — actual amount paid; feeds balance.
- balance — generated column: base_amount - salary_paid.
- notes — free-form text.
- UNIQUE (employee_id, period_id) — one entry per employee per month.

Special concerns:

- Open item: the service layer should validate that advance_id, when set, belongs to the same employee_id as the entry — the FK only guarantees the advance exists, not that it belongs to the right person.

7. `employee_advances`

```
CREATE TABLE employee_advances (
    advance_id      SERIAL PRIMARY KEY,
    employee_id     INT NOT NULL REFERENCES employees(employee_id),
    amount          NUMERIC(10,2) NOT NULL,
    date_given      DATE NOT NULL DEFAULT CURRENT_DATE,
    amount_deducted NUMERIC(10,2) NOT NULL DEFAULT 0,
    balance         NUMERIC(10,2) GENERATED ALWAYS AS (amount - amount_deducted) STORED,
    status          VARCHAR(20) DEFAULT 'open' CHECK (status IN ('open', 'cleared'))
);
```

Purpose: tracks money given to an employee ahead of payroll. Independent of any specific payroll_periods row — an advance can happen anytime.

Fields:

- advance_id — surrogate key, referenced by payroll_entries.advance_id.
- employee_id — required FK.
- amount — original amount advanced, required, no default.
- date_given — defaults to today via CURRENT_DATE, still required.
- amount_deducted — one of the two original bug fixes: missing NOT NULL in the first debugging pass, which could have silently broken the generated balance.
- balance — generated column: amount - amount_deducted.
- status — open/cleared, CHECK added mid-project.

Special concerns:

- None outstanding.

8. `recurring_bills`

```
CREATE TABLE recurring_bills (
    recurring_bill_id SERIAL PRIMARY KEY,
    vendor_id       INT NOT NULL REFERENCES vendors(vendor_id),
    description     VARCHAR(200) NOT NULL,
    expected_amount NUMERIC(10,2),
    frequency       VARCHAR(20) NOT NULL DEFAULT 'monthly'
                        CHECK (frequency IN ('monthly', 'quarterly', 'yearly')),
    is_active       BOOLEAN DEFAULT TRUE,
    created_at      TIMESTAMP DEFAULT now()
    -- created_by, updated_by, updated_at added later via auth_audit_schema.sql, for parity
);
```

Purpose: an entirely new table representing an ongoing obligation itself (the vehicle EMI, the electricity bill) as a standing row, separate from any individual payment. Fixes the original gap where each month's EMI payment had nothing formally connecting it to the last.

Fields:

- recurring_bill_id — surrogate key, referenced by vendor_transactions.recurring_bill_id.
- vendor_id — required FK.
- description — required (unlike vendor_transactions.description) — this row IS the thing being identified.
- expected_amount — informational only, not enforced against actual transaction amounts.
- frequency — monthly/quarterly/yearly, CHECK-constrained from the start — a genuine three-way choice, unlike the binary status fields elsewhere.
- is_active — soft-delete flag.
- created_at — added on request; judged a real financial commitment, not just a lookup list.
- created_by / updated_by / updated_at — added later to bring this table to full parity with the other five owned financial tables.

Special concerns:

- Originally missing its audit trigger — caught when a live test query against audit_log showed 13 rows across five tables, with recurring_bills conspicuously absent. trg_audit_recurring_bills was added afterward, along with the created_by/updated_by/updated_at columns. Any new financial table added in a future module needs this same two-part treatment — it doesn't happen automatically just by creating the table.

9. `vendor_transactions`

```
CREATE TABLE vendor_transactions (
    transaction_id  SERIAL PRIMARY KEY,
    vendor_id       INT NOT NULL REFERENCES vendors(vendor_id),
    recurring_bill_id INT REFERENCES recurring_bills(recurring_bill_id),
    description     VARCHAR(200),
    amount          NUMERIC(10,2) NOT NULL,
    paid_amount     NUMERIC(10,2) NOT NULL DEFAULT 0,
    balance         NUMERIC(10,2) GENERATED ALWAYS AS (amount - paid_amount) STORED,
    transaction_date DATE NOT NULL DEFAULT CURRENT_DATE,
    reference_no    VARCHAR(60)
);
```

Purpose: replaces the old spreadsheet's flat one-balance-per-vendor row with real transaction history — every bill or payment gets its own row.

Fields:

- transaction_id — surrogate key.
- vendor_id — required FK.
- recurring_bill_id — nullable FK linking a payment back to its ongoing obligation.
- description — optional free text for this specific transaction.
- amount, paid_amount — required amounts; paid_amount feeds balance.
- balance — generated column: amount - paid_amount.
- transaction_date — defaults to today, still required.
- reference_no — bill/vehicle number, free text.

Special concerns:

- is_recurring BOOLEAN was removed when recurring_bill_id was added — keeping both would have let a row claim is_recurring=true while recurring_bill_id was NULL. "Is this recurring?" is now simply recurring_bill_id IS NOT NULL — one source of truth.

10. `director_draws`

```
CREATE TABLE director_draws (
    draw_id         SERIAL PRIMARY KEY,
    director_id     INT NOT NULL REFERENCES directors(director_id),
    period_id       INT NOT NULL REFERENCES payroll_periods(period_id),
    previous_balance NUMERIC(10,2) NOT NULL DEFAULT 0,
    new_draw        NUMERIC(10,2) NOT NULL DEFAULT 0,
    paid_amount     NUMERIC(10,2) NOT NULL DEFAULT 0,
    balance         NUMERIC(10,2) GENERATED ALWAYS AS (previous_balance + new_draw -
paid_amount) STORED,
    notes           TEXT,
    UNIQUE (director_id, period_id)
);
```

Purpose: a director's personal draw for a given month — structurally similar to payroll_entries, kept separate since a draw isn't a salary.

Fields:

- draw_id — surrogate key.
- director_id, period_id — required FKs.

- previous_balance — carried forward from the prior period; correctly NOT NULL from the start.
- new_draw, paid_amount — this period's new amount and what's been paid.
- balance — generated column with three terms: previous_balance + new_draw - paid_amount — the first generated column in the schema with more than two terms.
- notes — added for parity with payroll_entries.
- UNIQUE (director_id, period_id) — one draw record per director per month.

Special concerns:

- None outstanding.

11. loans

```
CREATE TABLE loans (
    loan_id         SERIAL PRIMARY KEY,
    party_type      VARCHAR(20) NOT NULL CHECK (party_type IN
('employee','director','vendor')),
    party_id        INT NOT NULL,
    principal_amount NUMERIC(10,2) NOT NULL,
    interest_amount NUMERIC(10,2) NOT NULL DEFAULT 0,
    paid_amount     NUMERIC(10,2) NOT NULL DEFAULT 0,
    balance         NUMERIC(10,2) GENERATED ALWAYS AS (principal_amount + interest_amount -
paid_amount) STORED,
    start_date      DATE NOT NULL DEFAULT CURRENT_DATE,
    status          VARCHAR(20) NOT NULL DEFAULT 'active'
                        CHECK (status IN ('active', 'closed', 'defaulted')),
    notes           TEXT
);
```

Purpose: the schema's one polymorphic table — a single shared table for a loan held by any of the three party types, since a loan record's shape is identical regardless of who holds it.

Fields:

- loan_id — surrogate key.
- party_type — employee/director/vendor, CHECK-constrained from the start.
- party_id — the one column in the entire schema with no real REFERENCES clause; enforced instead by the fn_validate_loan_party trigger, which checks the right table based on party_type.
- principal_amount — required, no default.
- interest_amount, paid_amount — the second of the two original bug fixes, both missing NOT NULL in the first debugging pass.
- balance — generated column: principal_amount + interest_amount - paid_amount.
- start_date — defaults to today, still required.
- status — active/closed/defaulted, added mid-project — three real states, a good fit for CHECK-constrained text rather than a boolean.
- notes — same free-form pattern as elsewhere.

Special concerns:

- 'defaulted' is a judgment call, not derivable from data — this schema has no due-date or installment-schedule tracking, so marking a loan defaulted must be a manual action.
- Open item, deferred to the service layer: nothing here automatically flips status to 'closed' when balance reaches 0 — a comment in the SQL flags this for loan_service.py.

- Director financial privacy: rows where party_type='director' are restricted to superadmin only — enforced in loan_service.py, not the database, since loans is polymorphic and can't carry a route-level role restriction the way the dedicated /directors endpoints can.

12. `users`

```
CREATE TABLE users (
    user_id         SERIAL PRIMARY KEY,
    username        VARCHAR(50) UNIQUE NOT NULL,
    full_name       VARCHAR(150) NOT NULL,
    email           VARCHAR(150) UNIQUE NOT NULL,
    password_hash   TEXT NOT NULL,
    role            VARCHAR(30) NOT NULL DEFAULT 'data_entry'
                        CHECK (role IN ('superadmin', 'admin', 'accountant', 'data_entry',
'viewer')),
    is_active       BOOLEAN DEFAULT TRUE,
    last_login_at   TIMESTAMP,
    created_at      TIMESTAMP DEFAULT now()
);
```

Purpose: system access, not staff membership — a different kind of "who" than employees. Kept as a separate table since "who works here" and "who can log in" are different questions.

Fields:

- user_id — surrogate key, referenced by every created_by/updated_by column across the schema, and by audit_log.changed_by.
- username — UNIQUE, required login identifier.
- full_name — required.
- email — upgraded mid-project to UNIQUE NOT NULL.
- password_hash — stores only the bcrypt/argon2 hash, never a raw password. TEXT rather than VARCHAR(n) since hashes don't map neatly to a small length limit.
- role — five flat values. Defaults to the least-privileged role as a deliberate safety choice — a forgotten assignment fails safe, not over-privileged.
- is_active — deactivation matters more here than anywhere else, since deleting a user outright would orphan a large amount of historical accountability data.
- last_login_at — nullable, no default; stays NULL until the login endpoint sets it.
- created_at — standard auto-stamped pattern.

Special concerns:

- role's CHECK constraint was added mid-project (originally unconstrained free text).
- A fifth role, superadmin, was added and finalized: has every ability admin has, PLUS exclusive access to directors data (directors, director_draws, and any loans row where party_type='director'). admin is deliberately excluded from directors data specifically — the one place the hierarchy reverses. Implemented via ADMIN_ROLES/DIRECTOR_ROLES constants in dependencies.py rather than hand-typed role lists per route.

13. `audit_log`

```
CREATE TABLE audit_log (
    audit_id        BIGSERIAL PRIMARY KEY,
    table_name      VARCHAR(50) NOT NULL,
    record_id       INT NOT NULL,
    action          VARCHAR(10) NOT NULL CHECK (action IN ('INSERT','UPDATE','DELETE')),
    changed_by      INT REFERENCES users(user_id),
    changed_at      TIMESTAMP DEFAULT now(),
    old_values      JSONB,
    new_values      JSONB
);
```

Purpose: the governance layer sitting on top of everything else — not describing money, but accountability for the money. Populated automatically by database triggers, not application code.

Fields:

- audit_id — the only BIGSERIAL in the schema; this table gains a row on every change to five other tables combined, so it accumulates far faster than anything else.
- table_name — which table changed. Deliberately not a foreign key or CHECK-constrained list.
- record_id — the changed row's primary key value. Also deliberately not a foreign key — an audit log must keep working after the row it describes is gone.
- action — INSERT/UPDATE/DELETE, CHECK-constrained to the three values TG_OP can ever hold.
- changed_by — the one real foreign key on this table, to users.
- changed_at — same auto-stamped pattern as created_at elsewhere.
- old_values, new_values — the only JSONB columns in the schema; each stores the entire row, before and after, via to_jsonb(OLD)/to_jsonb(NEW).

Special concerns:

- table_name has no CHECK/FK constraining it to real table names — confirmed as the right call: action's valid set is permanently fixed, but table_name's valid set grows every time a new table gets an audit trigger. A CHECK here would need updating in lockstep, or every write to a newly-audited table would fail outright. A soft-guardrail comment was added instead, documenting that this column should only be populated by fn_audit_trigger.

Part 2 — The Schema's Three-Tier Hierarchy

Beyond the entity relationships, the schema has a structural hierarchy worth understanding on its own — it's also the order tables must be created in, and the order Alembic migrations need to respect.

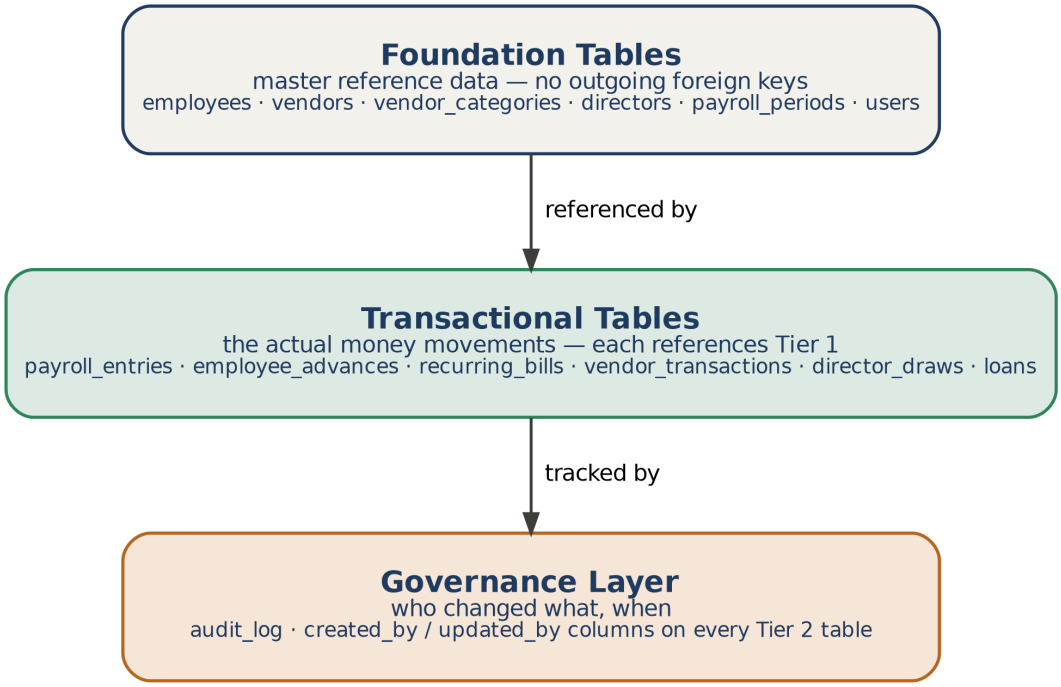

Figure 1 — the three-tier hierarchy.

Tier 1 — Foundation tables

employees, vendors, vendor_categories, directors, payroll_periods, users. No foreign keys pointing out of them — pure reference data: who exists, and which calendar month is being discussed. Everything else traces back to one of these six.

Tier 2 — Transactional tables

payroll_entries, employee_advances, recurring_bills, vendor_transactions, director_draws, loans. Every one has at least one foreign key into Tier 1 — recording something that happened, tied to a specific employee/vendor/director and (for payroll and director draws) a specific period. loans is the exception: its link to Tier 1 is trigger-enforced, not a real foreign key, since it can point at three different Tier 1 tables depending on party_type.

Tier 3 — Governance layer

audit_log, plus the created_by/updated_by/updated_at columns on every Tier 2 table. Doesn't describe money — describes accountability for the money. audit_log has one real link back into Tier 1 (changed_by to users), but its other identifying fields are deliberately plain values, not foreign keys, so it keeps working even after the row it describes is gone.

Why this matters practically: this is the exact order tables must be created in (Tier 1, then Tier 2, then Tier 3) — the same order Alembic migrations need to respect, since Postgres rejects a REFERENCES clause pointing at a table that doesn't exist yet. It's also a useful checklist when deciding where a new column belongs: does it describe who/what (Tier 1), what happened (Tier 2), or who's accountable (Tier 3)?

Part 3 — Entity-Relationship Diagram

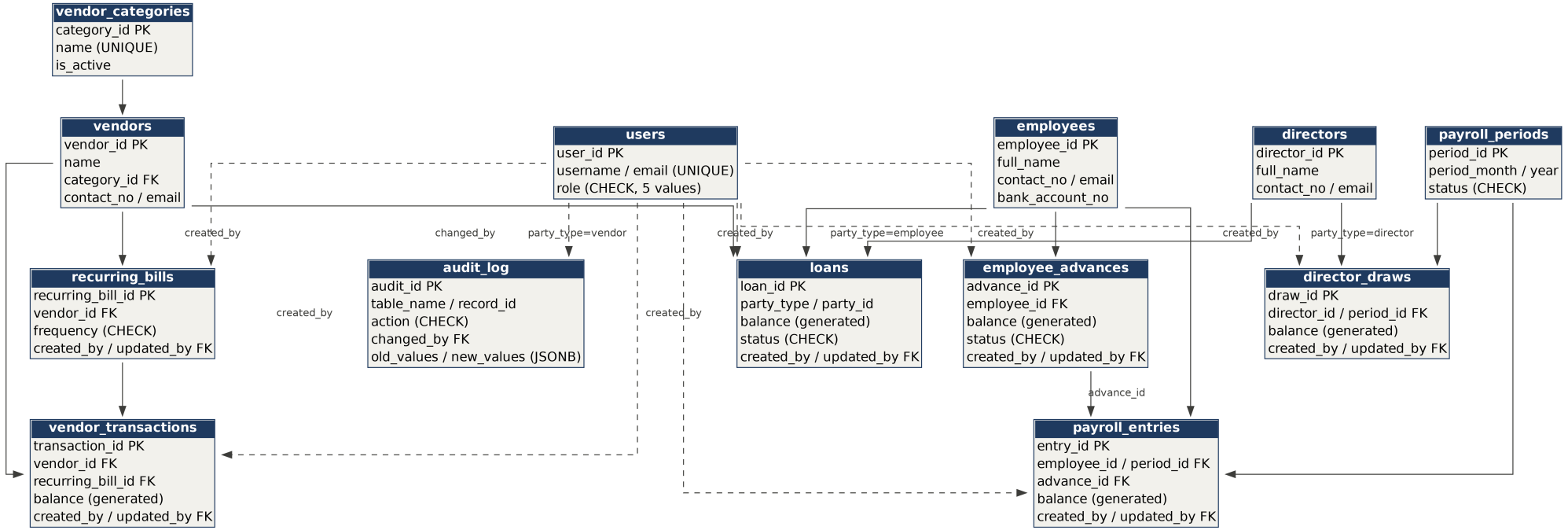

Figure 2 — all 13 tables and their relationships. Dashed lines are created_by/changed_by links to users; solid lines are structural foreign keys. loans' links to employees, directors, and vendors are trigger-validated (fn_validate_loan_party), not real foreign keys, since Postgres cannot point one FK at three different tables.